

BDBT – Obsługa błędów w aplikacji webowej

Tomasz Mrozek
Instytut Telekomunikacji
Wydział Elektroniki i Technik Informacyjnych



Zakres prezentacji

- Obsługa wybranych błędów w aplikacji webowej

1

OBSŁUGA BŁĘDÓW

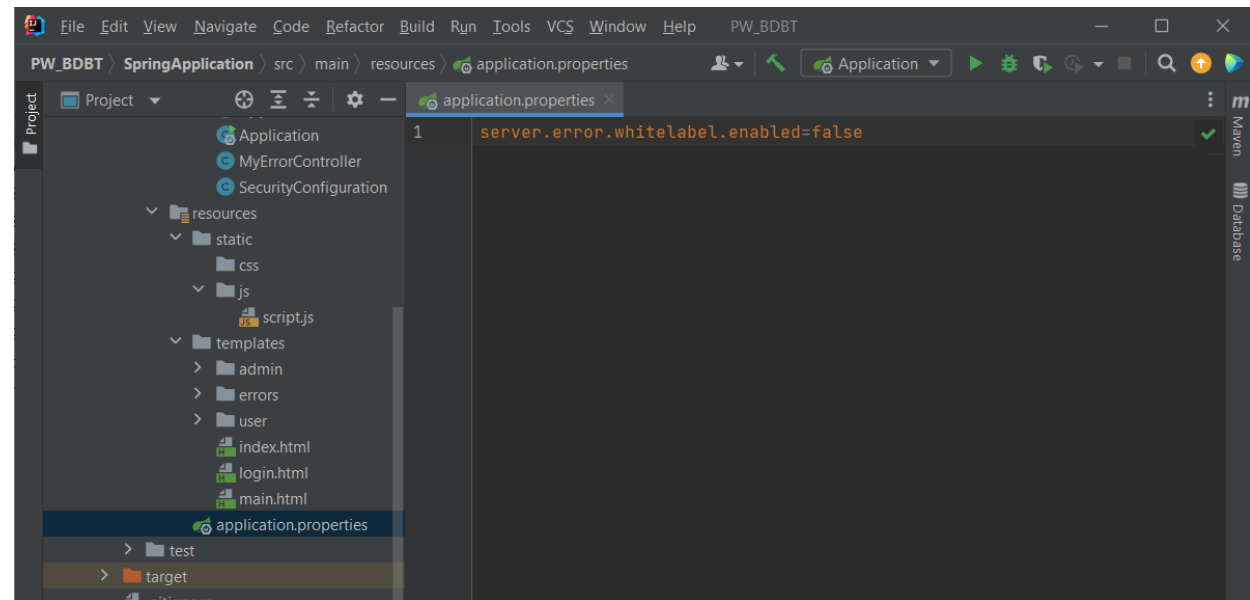
Obsługa błędów – wpis w application.properties

Przejsć do pliku „application.properties” i dodać wpis: **server.error.whitelabel.enabled=false**.

Dodanie tej opcji spowoduje, że w przypadku wystąpienia błędu nie wyświetli się biała strona tzw. „whitelabel error page”.

Zamiast tej strony, będą wyświetlane informacje o błędzie obsługiwane przez wykorzystywane przez Nas narzędzia, np. wbudowany serwer Tomcat. Błąd będzie wyświetlany w postaci krótkiej informacji z kodem błędu i jego opisem.

W ramach testu uruchom aplikację i wywołaj błąd (oczekiwany wynik został przedstawiony na obrazie z prawej strony).



HTTP Status 404 – Not Found

Type Status Report

Message /BookStoreWebsite/view_books

Description The origin server did not find a current representation for the target resource or is not willing to disclose that one exists.

Apache Tomcat/9.0.16

Obsługa błędów – stworzenie kontrolera błędów

Kolejnym krokiem jest dodanie kontrolera do obsługi błędów.

W zaprezentowanym przykładzie aplikacja będzie obsługiwać następujące błędy:

403 – FORBIDDEN,

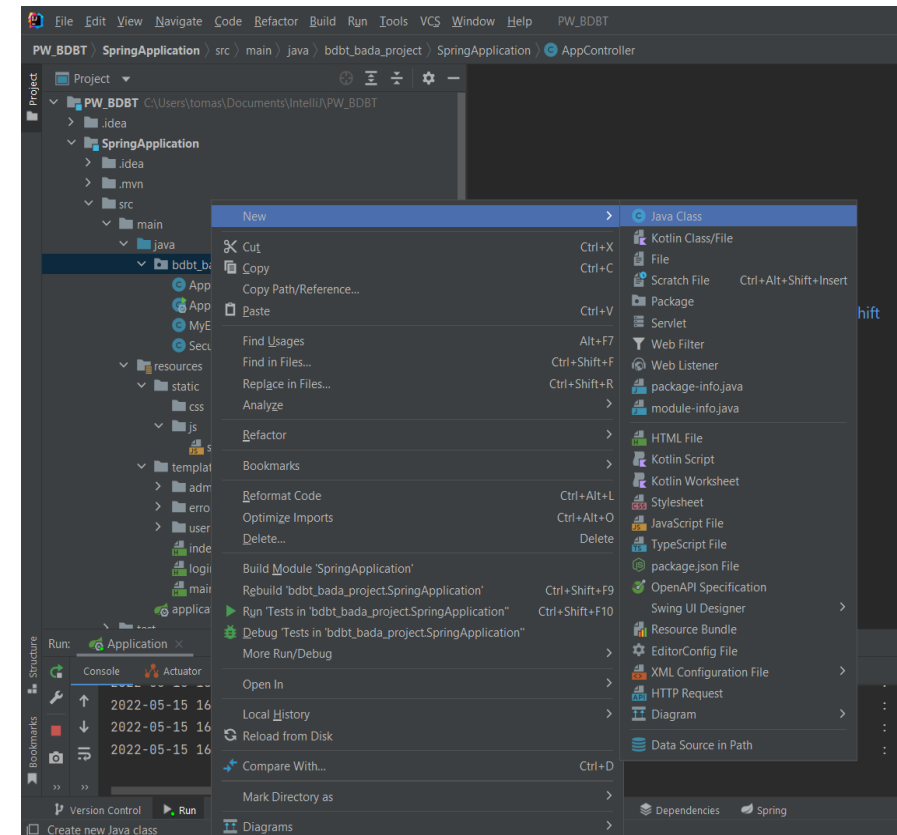
404 – NOT_FOUND,

500 – INTERNAL_SERVER_ERROR,

504 – GATEWAY_TIMEOUT,

Inne nie zdefiniowane.

Klikamy prawym klawiszem myszy w folder projektu (tutaj: `bdbt_bada_project.SpringApplication`) i tworzymy nowy plik o nazwie np. `MyErrorController`.



Obsługa błędów – stworzenie kontrolera błędów

W stworzonym pliku tworzymy klasę MyErrorController.

```
public class MyErrorController {}
```

Do klasy dodajemy rozszerzenie „implements ErrorController”, oraz adnotację @Controller (można to dodać automatycznie najeżdżając kursorem na podświetlony fragment kodu jako błąd i wybrać sugerowane działanie).

```
@Controller  
public class MyErrorController implements ErrorController { }
```

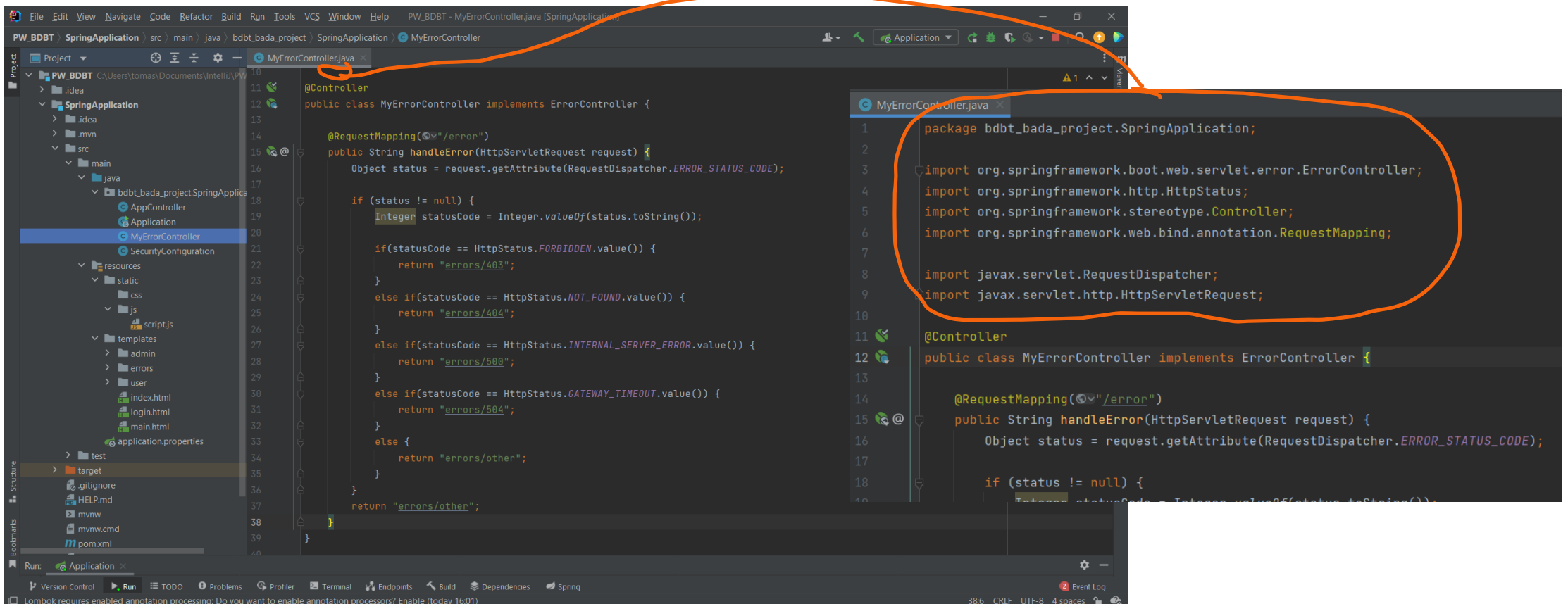
Następnie w stworzonej klasie tworzymy metodę handleError wraz z adnotacją @RequestMapping(„/error”).

```
@RequestMapping("/error")  
public String handleError(HttpServletRequest request) { }
```

Zawartość stworzonej metody została przedstawiona na kolejnej stronie.

Obsługa błędów – stworzenie kontrolera błędów

Zawartość pliku i klasy MyErrorController (nie zapomnieć o imporcie bibliotek).



```
package bdbt_bada_project.SpringApplication;

import org.springframework.boot.web.servlet.error.ErrorController;
import org.springframework.http.HttpStatus;
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.RequestMapping;

import javax.servlet.RequestDispatcher;
import javax.servlet.http.HttpServletRequest;

@Controller
public class MyErrorController implements ErrorController {

    @RequestMapping("/error")
    public String handleError(HttpServletRequest request) {
        Object status = request.getAttribute(RequestDispatcher.ERROR_STATUS_CODE);

        if (status != null) {
            Integer statusCode = Integer.valueOf(status.toString());

            if (statusCode == HttpStatus.FORBIDDEN.value()) {
                return "errors/403";
            }
            else if (statusCode == HttpStatus.NOT_FOUND.value()) {
                return "errors/404";
            }
            else if (statusCode == HttpStatus.INTERNAL_SERVER_ERROR.value()) {
                return "errors/500";
            }
            else if (statusCode == HttpStatus.GATEWAY_TIMEOUT.value()) {
                return "errors/504";
            }
            else {
                return "errors/other";
            }
        }
        return "errors/other";
    }
}
```

Obsługa błędów – stworzenie kontrolera błędów

Kod dla pliku **MyErrorController.java** (do skopiowania cz. 1):

```
package bdbt_bada_project.SpringApplication;

import org.springframework.boot.web.servlet.error.ErrorController;
import org.springframework.http.HttpStatus;
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.RequestMapping;

import javax.servlet.RequestDispatcher;
import javax.servlet.http.HttpServletRequest;

@Controller
public class MyErrorController implements ErrorController {
```


Obsługa błędów – stworzenie kontrolera błędów

Kod dla pliku **MyErrorController.java** (do skopiowania cz. 2):

```
@RequestMapping("/error")
public String handleError(HttpServletRequest request) {
    Object status = request.getAttribute(RequestDispatcher.ERROR_STATUS_CODE);

    if (status != null) {
        Integer statusCode = Integer.valueOf(status.toString());

        if(statusCode == HttpStatus.FORBIDDEN.value()) {
            return "errors/403";
        }
        else if(statusCode == HttpStatus.NOT_FOUND.value()) {
            return "errors/404";
        }
    }
}
```

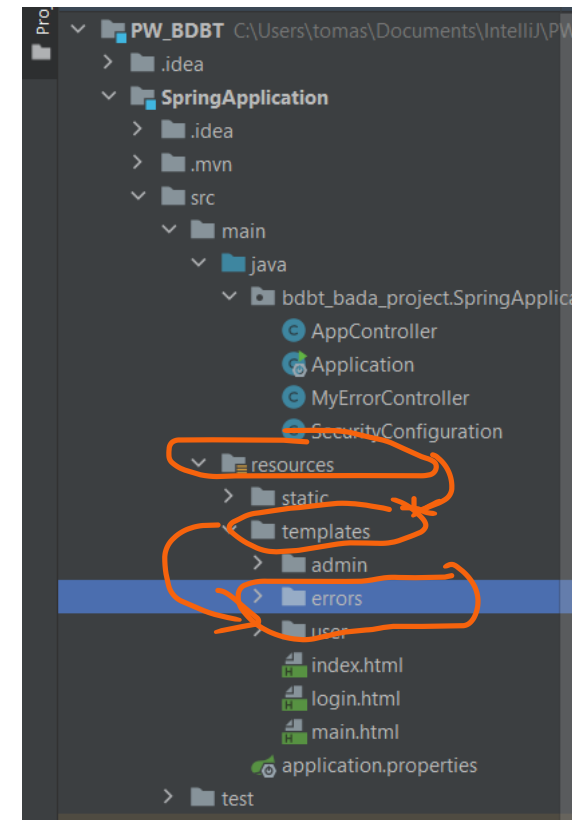
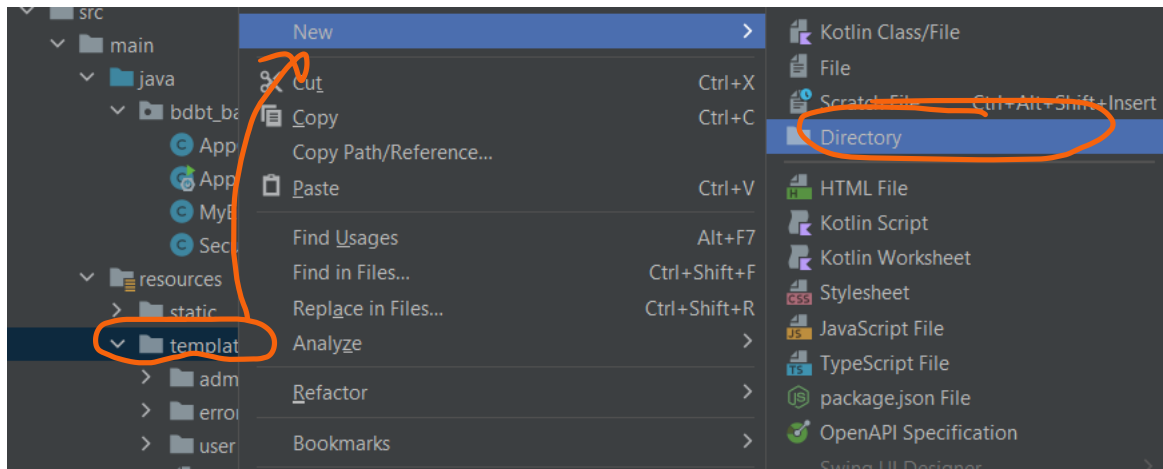
Obsługa błędów – stworzenie kontrolera błędów

Kod dla pliku **MyErrorController.java** (do skopiowania cz. 3):

```
    else if(statusCode == HttpStatus.INTERNAL_SERVER_ERROR.value()) {  
        return "errors/500";  
    }  
    else if(statusCode == HttpStatus.GATEWAY_TIMEOUT.value()) {  
        return "errors/504";  
    }  
    else {  
        return "errors/other";  
    }  
    }  
    return "errors/other";  
    }  
}
```

Obsługa błędów – stworzenie folderu oraz plików .html

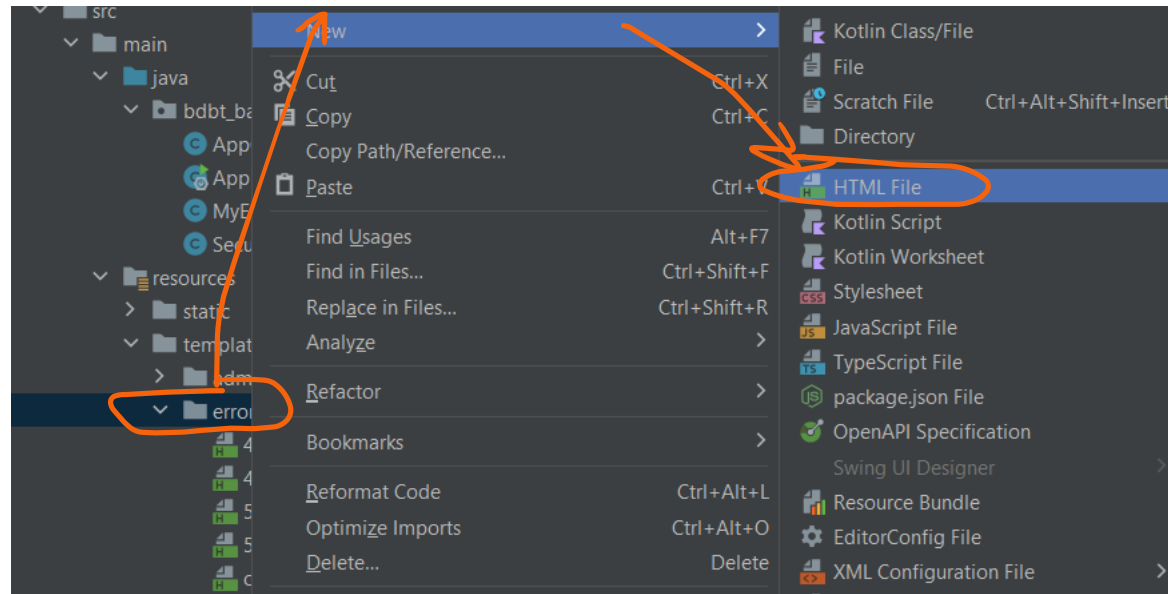
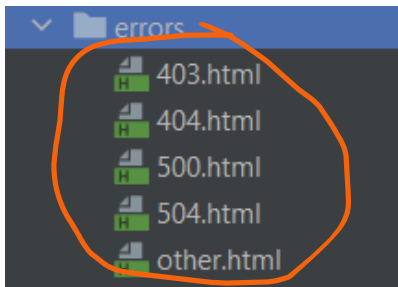
Przed ostatnim etapem jest stworzenie folderu **errors** w lokalizacji **resources/templates**



Obsługa błędów – stworzenie folderu oraz plików .html

W ostatnim etapie, w stworzonym folderze **errors** należy stworzyć pliki .html, które będą odpowiedzialne za obsługę konkretnych błędów. Należy stworzyć następujące pliki:

- 403.html,
- 404.html,
- 500.html,
- 504.html,
- other.html.



Obsługa błędów – stworzenie folderu oraz plików .html

Przykładowa zawartość pliku 403.html (część 1):

```
MyErrorController.java x 403.html x
1  <!DOCTYPE HTML>
2  <html lang="en" xmlns:th="http://www.thymeleaf.org">
3  <head>
4      <meta charset="UTF-8"/>
5      <title>Error Page</title>
6
7      <link th:rel="stylesheet" th:href="@{/webjars/bootstrap/4.0.0-2/css/bootstrap.min.css}" />
8  </head>
9  <body>
10
11      <!-- Navigation -->
12      <nav class="navbar navbar-expand-lg navbar-dark bg-dark static-top">
13          <div class="container">
14              <a class="navbar-brand" href="/">BDBT Spring Boot Project - Part II</a>
15              <button class="navbar-toggler" type="button" data-toggle="collapse" data-target="#navbarResponsive"
16                  aria-controls="navbarResponsive"
17                  aria-expanded="false" aria-label="Toggle navigation">
18                  <span class="navbar-toggler-icon"></span>
19              </button>
20          </div>
21      </nav>
22
23      <div class="container">
24
25          <div class="row">
26              <div class="col-md-4 mt-5">
```

Obsługa błędów – stworzenie folderu oraz plików .html

Przykładowa zawartość pliku 403.html (część 2):

```
MyErrorController.java x 403.html x
41 </nav>
42
43 <div class="container">
44
45     <div class="row">
46
47         <div class="col-md-4 mt-5">
48
49             <form method="post" th:action="@{/index}">
50
51                 <div class="form-group">
52
53                     <div><b><p style="font-size:30px">Error</p></b></div>
54                     <div><p style="font-size:24px">403 - Forbidden page</p></div>
55                 </div>
56
57                 <button class="btn btn-primary" type="submit" value="Sign In">Back to home page</button>
58             </form>
59         </div>
60     </div>
61 </div>
62
63 <script th:src="@{/webjars/jquery/jquery.min.js}"></script>
64 <script th:src="@{/webjars/popper.js/umd/popper.min.js}"></script>
65 <script th:src="@{/webjars/bootstrap/js/bootstrap.min.js}"></script>
66
67 </body>
68 </html>
```

Obsługa błędów – stworzenie folderu oraz plików .html

Kod dla pliku **403.html** (do skopiowania cz. 1):

```
<!DOCTYPE HTML>
<html lang="en" xmlns:th="http://www.thymeleaf.org">
<head>
  <meta charset="UTF-8"/>
  <title>Error Page</title>

  <link th:rel="stylesheet" th:href="@{/webjars/bootstrap/4.0.0-2/css/bootstrap.min.css}" />
</head>
<body>
```

Obsługa błędów – stworzenie folderu oraz plików .html

Kod dla pliku **403.html** (do skopiowania cz. 2):

```
<!-- Navigation -->
<nav class="navbar navbar-expand-lg navbar-dark bg-dark static-top">
  <div class="container">
    <a class="navbar-brand" href="/">BDBT Spring Boot Project - Part II</a>
    <button class="navbar-toggler" type="button" data-toggle="collapse" data-target="#navbarResponsive"
      aria-controls="navbarResponsive"
      aria-expanded="false" aria-label="Toggle navigation">
      <span class="navbar-toggler-icon"></span>
    </button>
  </div>
</nav>
```


Obsługa błędów – stworzenie folderu oraz plików .html

Kod dla pliku **403.html** (do skopiowania cz. 3):

```
<div class="container">
  <div class="row">
    <div class="col-md-4 mt-5">

      <form method="post" th:action="@{/index}">

        <div class="form-group">
          <div><b><p style="font-size:30px">Error</p></b></div>
          <div><p style="font-size:24px">403 - Forbidden page</p></div>
        </div>

        <button class="btn btn-primary" type="submit" value="Sign In">Back to home page</button>
      </form>
    </div>
  </div>
</div>
```

Obsługa błędów – stworzenie folderu oraz plików .html

Kod dla pliku **403.html** (do skopiowania cz. 4):

```
<script th:src="@{/webjars/jquery/jquery.min.js}"></script>  
<script th:src="@{/webjars/popper.js/umd/popper.min.js}"></script>  
<script th:src="@{/webjars/bootstrap/js/bootstrap.min.js}"></script>  
  
</body>  
</html>
```

Po wypełnieniu pliku powyższym kodem w celu jego automatycznego uporządkowania (zapewnienia odpowiednich wcięć) możesz użyć skrótu **ctrl+alt+l** (l to małe L).

Obsługa błędów – stworzenie folderu oraz plików .html

W analogiczny sposób wypełnij kodem pozostałe pliki: 404.html, 500.html, 504.html oraz other.html.

Jedyną różnicą w pozostałych plikach jest zmiana informacji o wyświetlanym błędzie (linijka 31):

```
31      <div><p style="font-size:24px">403 - Forbidden page</p></div>
```

Dla pliku:

404.html, >404 - Page not found<

500.html: >500 – Internal Server Error<

504.html: >504 – Gateway Timeout<

others.html: >An unexpected error has occurred<

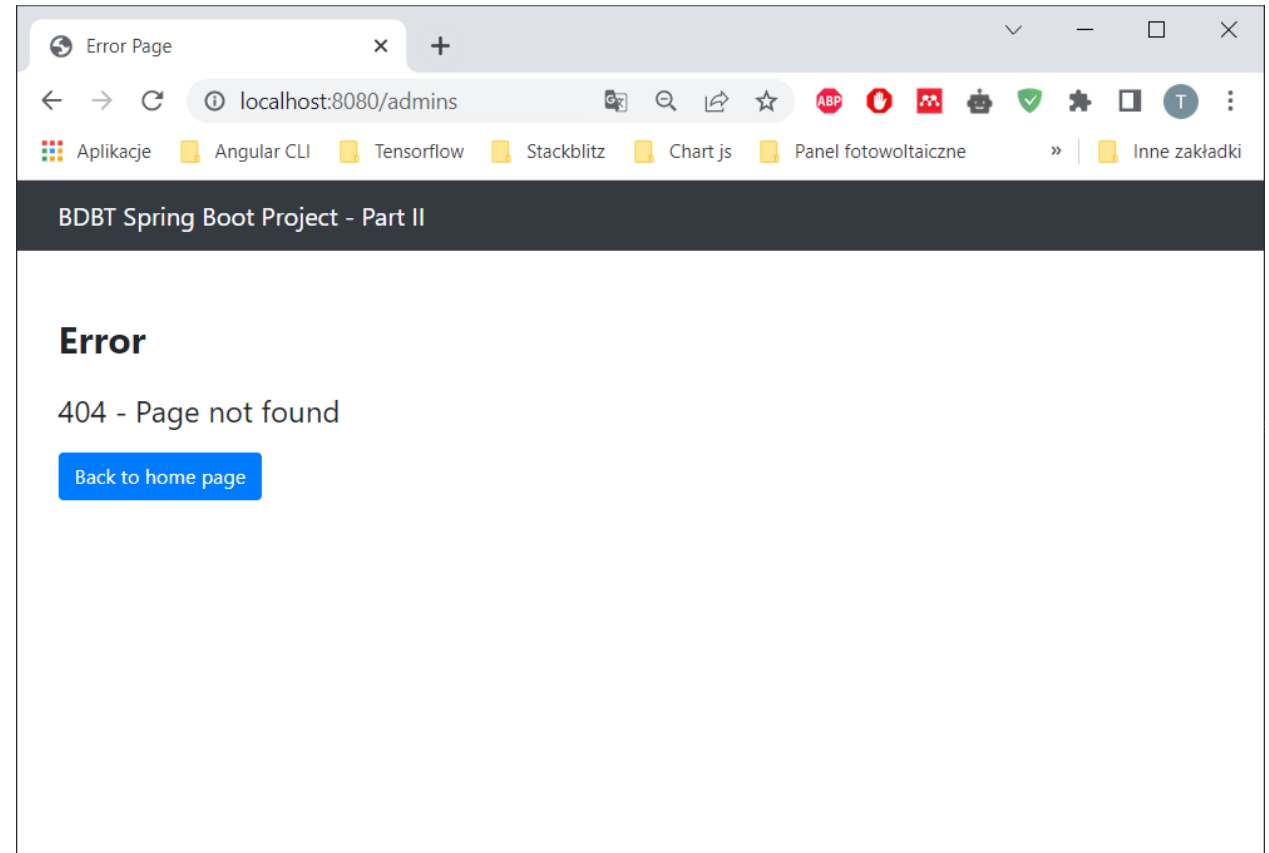
Obsługa błędów – prezentacja działania

Przykład 1:

Uruchomić aplikację i w adresie **localhost:8080** dodać **/admins**.

Podstrona „admins” nie istnieje dlatego pojawił się komunikat „404 – Page not found”.

Kliknięcie w button „Back to home page” spowoduje powrót do poprzedniej strony.



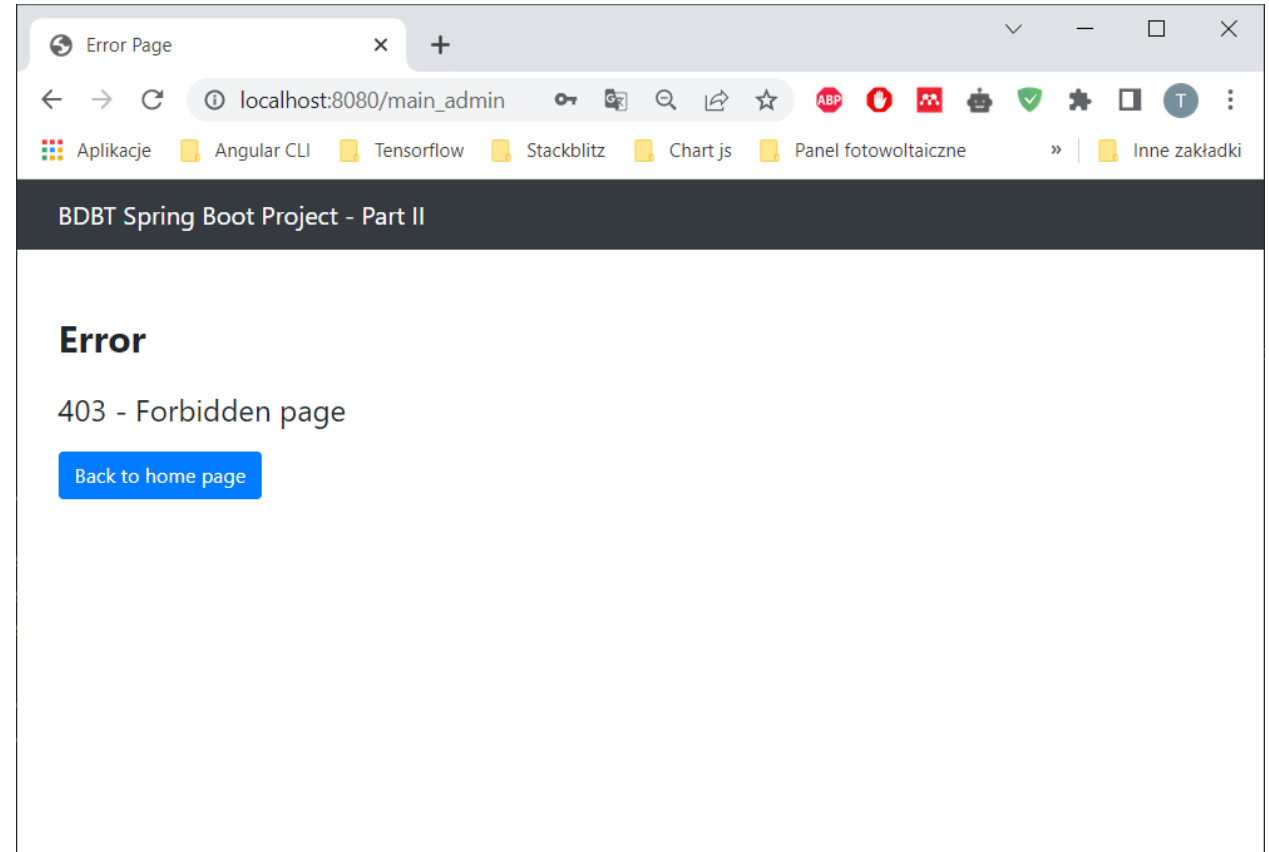
Obsługa błędów – prezentacja działania

Przykład 2:

Uruchomić aplikację i w adresie **localhost:8080** dodać **/main_admin**.

Wyświetli się strona logowania, gdzie w login i hasło wpisać „user/user”. Nastąpiła próba zalogowania na stronę przeznaczoną dla administratora z konta zwykłego użytkownika, dlatego pojawił się komunikat „403 – Forbidden page”.

UWAGA: Przykład ten jest do zrealizowania/uzyskania dopiero stworzeniu perspektyw użytkowników (plik: ..tworzenie_aplikacji_2_perspektywy.pdf).



Obsługa błędów – podsumowanie

Przeanalizować kod oraz dostosować go do swojego rozwiązania (aplikacji).

**Politechnika
Warszawska**

Koniec

